

UVM Testbench Yazma Notlarım

Bu defter, her bileşeni kendi elimle yazarken öğrendiğim kalıpları, kuralları ve takıldığım noktaları biriktiriyor.

Altın kural: - "NE" (sinyaller/alanlar/genişlikler) -> design.sv'den gelir - "NASIL" (sınıf/clocking/makro iskeleti) -> UVM kalıbından gelir (sabit) - Object mi Component mi? -> constructor ve utils makrosu buna göre değişir

Object vs Component (her yerde lazım):

	Object (seq_item, sequence)	Component (sequencer, driver, monitor, agent, env, test)
extends	uvm_object / uvm_sequence / uvm_sequence_item	uvm_component / uvm_driver / ...
utils makrosu	uvm_object_utils	uvm_component_utils
constructor	(string name)	(string name, uvm_component parent)
super.new	super.new(name)	super.new(name, parent)
faz (build/run)	yok (body var)	var

BÖLÜM 1 — INTERFACE

Interface = DUT ile testbench arasındaki "kablo demeti". Sadece DUT'un PORT LISTESİNİ yansıtır; DUT'un iç mantığını (always blokları) İCERMEZ.

```
interface <isim> (input logic clk, reset);
    logic <genislik> <sinyal>;          // DUT portlari (clk/reset haric)

    clocking driver_cb @(posedge clk);  // DRIVER: DUT girisi->output, cikisi->input
        default input #1 output #1;
        output <sinyal>;
        input <sinyal>;
    endclocking

    clocking monitor_cb @(posedge clk); // MONITOR: hepsi input
        default input #1 output #1;
        input <sinyal>;
    endclocking

    modport DRIVER (clocking driver_cb, input clk, reset);
    modport MONITOR (clocking monitor_cb, input clk, reset);
endinterface
```

Yon kurali: DUT'ta `input` -> driver_cb'de `output`; DUT'ta `output` -> driver_cb'de `input`. Monitor'da HEP `input`. (Ayni telin iki ucu; driver suren taraf.)

BÖLÜM 2 — SEQUENCE ITEM (transaction)

Bir islemi tarif eden VERİ PAKETİ. OBJECT'tir.

```
class <isim> extends uvm_sequence_item;
    rand bit <genislik> <alan>; // testbench uretiyor (DUT girisi)
    bit <genislik> <alan>; // DUT donduruyor (DUT cikisi) -> rand DEGIL

    `uvm_object_utils_begin(<isim>)
        `uvm_field_int(<alan>, UVM_ALL_ON)
    `uvm_object_utils_end

    function new(string name = "<isim>");
        super.new(name);
    endfunction

    constraint <isim>_c { <kural>; }
endclass
```

Onemli kurallar: - rand mi? "Ben mi uretip veriyorum (rand) yoksa DUT mu donduruyor (rand degil)". Kisayol: design.sv'de INPUT -> rand;

BÖLÜM 3 — SEQUENCER

Sequence'in urettigi paketleri driver'a akitan "trafik memuru". COMPONENT. En kısa dosya; icine mantik yazilmaz (temel uvm_sequencer her isi yapar).

```
class <isim> extends uvm_sequencer #(<transaction tipin>);
  `uvm_component_utils(<isim>)

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
endclass
```

Nereden ne gelir: - #(<transaction>) -> hangi paketi tasir (kendi seq_item'in) - geri kalan -> sabit component kalibi - build_phase / run_phase YAZILMAZ.

BÖLÜM 4 — SEQUENCE

Asil SENARYOYU ureten parca. OBJECT'tir (component degil!). Icinde body() gorevi olur.

```
class <isim> extends uvm_sequence #(<transaction tipin>);
  `uvm_object_utils(<isim>)

  function new(string name = "<isim>");
    super.new(name);
  endfunction

  task body();
    repeat(<kac tane>) begin
      `uvm_do(req)          // uret + rastgelelestir + yolla (hepsi tek satir)
    end
  endtask
endclass
```

- req -> uvm_sequence icinde HAZIR gelir, ayrıca tanımlamaya gerek yok.
- uvm_do(req) -> paketi uretir, randomize eder, sequencer'a yollar.
- Benim vereceğim karar: senaryo (hangi islemler) + kac tane (repeat).

GENEL KAVRAM — Farkli bir DUT gelince ne degisir?

Her UVM dosyasi = SABIT ISKELET + DUT'A OZEL ICERIK. Farkli DUT gelince SADECE DUT'a ozel kisim degisir; iskelet AYNI kalir.

Bilesen	DUT'a ozel (DEGISIR)	Sabit iskelet (AYNI kalir)
Interface	sinyal isim/genislik/yon	clocking/modport yapisi, yon kurali
Transaction	hangi alanlar, genislik, constraint	class/utils/constructor iskeleti
Sequencer	sadece #() icindeki tip adi	GERI KALAN HER SEY (en evrensel)
Sequence	senaryo (islem, adet)	body/ uvm_do iskeleti
Driver	drive() protokolu/zamanlamasi	get_next_item/item_done, config_db
Monitor	hangi sinyal ne zaman orneklenir	analysis_port + dongu
Agent/Env/Test/Top	hangi tip/isim kullanildigi	neredeyse tamamı sabit

Driver ve Monitor en cok degisenler; Sequencer/Agent/Env neredeyse hic degismez.

"Parametrize etmek" != "Farkli DUT" (en kritik ayrim!)

- PARAMETRIZE: AYNI DUT'un BOYUT varyasyonu icindir. Ornek: yine bu memory ama ADDR_WIDTH 2->4 ya da DATA_WIDTH 8->16.

Parametre kullanınca genislikleri tek yerden degistirirsın. Yani parametrize = "ayni tasarim, farkli olcu". Mekanik: `interface mem_if # (parameter ADDR_WIDTH=2, DATA_WIDTH=8) (...)` ve genisliklerde `[ADDR_WIDTH-1:0]` yazarsın.

- FARKLI TUR DUT (FIFO, UART, ALU gibi): parametreyle HALLEDİLMEZ. DUT'a özel kisimlari (sinyaller, alanlar, driver protokolu, scoreboard modeli) BASTAN yazarsın -- ama iskelet gene AYNI kalir.

Yani "farkli DUT gelince neyi parametrize ederim?" sorusunun cevabi: Parametrize, farkli DUT'un araci DEGİL. Farkli DUT'ta parametre degil, DUT'a özel ICERİĞİ yeniden yazarsın (iskeleti kopyalarsın).

Bu memory icin ne yapmaliyim?

- Parametrize etmek ZORUNDA DEGİLİM. Genislikleri sabit ([1:0], [7:0]) yazabilirim.
- DUT'a özel kisimlari design.sv'ye bakarak yazarım; iskeleti kaliptan alirim.
- Ilerride farkli DUT gelirse: ayni iskeleti kullan, DUT'a özel kismi degistir.

DIKKAT — takildigim/hata yaptigim noktalar

- NOKTALI VIRGUL yorumdan ONCE olmalı: YANLIS: `rand bit wr_en // aciklama; (; yorumun icinde -> sayilmaz)` DOGRU : `rand bit wr_en; // aciklama`
- Her tanimin sonunda ; olmalı (rdata'yi unutmustum).
- SINIF ADI = FACTORY ADI (birebir): `class seq_item -> uvm_object_utils_begin(seq_item)` (ayni olmalı). "Sinif adi" = class kelimesinden hemen sonraki isim.
- Kalibimdaki `<...>` yer tutucudur; kod yazarken `< >` ISARETLERİNİ SİLMELİYİM.
- Dosya adinda `.sv` uzantisi olmalı; yoksa VSCode dosyayi TypeScript sanar ve gecarli kodu kirmiziyla cizer.
- VSCode'un kirmizi cizgileri SystemVerilog icin GUVENİLİR DEGİL. Gercek dogruluk testi = DSim'in derlemesi.

BÖLÜM 7 — Testbench'i Baştan Sona: Sıra, Kalıp ve Bizim Tasarımımız

Bu bölüm, `mem_vf` projesinde **sıfırdan kurup DSim'de UVM_ERROR: 0 ile çalıştırdığımız** testbench'in tamamını topluyor: her dosya ne iş yapıyor, hangi satır sabit kalıp, hangi satırı kendi DUT'umuza (memory) göre biz yazdık, ve testbench yazmaya nereden başlanır.

Not: Aşağıdaki kodlar, oturumumuzdaki **son çalışan** haline göreler. Kendi dosyalarınıza birebir karşılaştırmak istersen açıp bakabilirsiniz; mantık ve yapı bire bir budur.

7.0 — Nereden başlanır? Hiyerarşide AŞAĞIDAN YUKARI

En sık sorulan soru: "önce hangi dosyayı yazayım?" Cevap: **aşağıdan yukarı (bottom-up)**. Sebep basit — her katman, kendinden bir alttakine dayanır. Alt yoksa üst derlenmez.

Yazma sırası (bizim izlediğimiz sıra):

- design.sv** — DUT (zaten var, biz yazmadık; ama her şey ondan türer).
- interface (mem_if)** — DUT'un sinyalleri. Her şeyin dili buradan gelir.
- seq_item** — bir işlemi tarif eden veri paketi (transaction).
- sequence** — senaryo (hangi işlemler, kaç tane).
- sequencer** — paketleri driver'a akıtan trafik memuru.
- driver** — paketi alıp pinleri süren el.
- monitor** — pinleri izleyip paket toplayan göz.
- agent** — driver + monitor + sequencer'ı bir kutuda toplar.
- scoreboard** — beklenen ile geleni karşılaştıran hakem.
- env** — agent + scoreboard'u bir kutuda toplar.
- test** — env'i kurar, sequence'i başlatır.
- tbench_top** — DUT + interface + test'i birleştiren en üst modül.

Mantık: driver'ı yazabilmek için önce `seq_item` (taşıdığı paket) ve `interface` (süreceği pinler) lazım. agent'ı yazabilmek için driver/monitor/sequencer lazım. env için agent/scoreboard lazım. En üstte test ve top var. Yani "en bağımsız" parçadan (interface) "en bağımlı" parçaya (top) doğru gidilir.

7.1 — design.sv (DUT) — biz yazmadık, ama her şey buradan türer

```
module memory
#( parameter ADDR_WIDTH = 2,
  parameter DATA_WIDTH = 8 ) (
  input  clk,
  input  reset,
  input  [ADDR_WIDTH-1:0] addr,
  input  wr_en,
  input  rd_en,
  input  [DATA_WIDTH-1:0] wdata,
  output reg [DATA_WIDTH-1:0] rdata
);
reg [DATA_WIDTH-1:0] mem [2**ADDR_WIDTH];    // 4 gözlü bellek

always @(posedge reset)                      // reset kenarında
  for(int i=0;i<2**ADDR_WIDTH;i++) mem[i]=8'hFF; // hepsini FF yap

always @(posedge clk)
  if (wr_en) mem[addr] <= wdata;             // yazma

always @(posedge clk)
  if (rd_en) rdata <= mem[addr];             // OKUMA: 1 saat gecikmeli!
endmodule
```

Bu dosyadan öğrendiğimiz, tüm testbench'i belirleyen **gerçekler**:

- Sinyaller ve genişlikler: `addr[1:0]`, `wr_en`, `rd_en`, `wdata[7:0]`, `rdata[7:0]`. (Interface ve seq_item bunları birebir yansıtır.)
- Bellek 4 gözlü (`2**ADDR_WIDTH = 4`), reset'te hepsi `8'hFF`. (Scoreboard'un `sc_mem`'i de 4 göz + FF ile başlar.)
- **Okuma register'lı, 1 saat gecikmeli** (`rdata <= mem[addr]`). Bu tek satır, monitörün okuma zamanlamasını belirledi (bkz. 7.13 debug).

Bizim tek RTL müdahalemiz: ilk derlemede `rdata` hem port listesinde hem de gövdede ayrı `reg` olarak tanımlıydı → `PortRedefinition` hatası. Çözüm: port listesinde `output reg [DATA_WIDTH-1:0] rdata` yapıp gövdedeki ayrı `reg rdata;` satırını sildik. (RTL mantığı değişmedi, sadece tek tanım haline getirdik.)

7.2 — mem_if.sv (interface) — DUT ile testbench arasındaki kablo demeti

```

interface mem_if (input logic clk, reset);
    logic [1:0] addr;
    logic      wr_en;
    logic      rd_en;
    logic [7:0] wdata;
    logic [7:0] rdata;

    clocking driver_cb @(posedge clk);      // DRIVER saati
        default input #1 output #1;
        output addr;                        // DUT'un GİRİŞİ -> driver ÇIKIŞ
        output wr_en;
        output rd_en;
        output wdata;
        input  rdata;                        // DUT'un ÇIKIŞI -> driver GİRİŞ
    endclocking

    clocking monitor_cb @(posedge clk);     // MONITOR saati
        default input #1 output #1;
        input  addr;                        // MONITOR: HEPSİ input (sadece izler)
        input  wr_en;
        input  rd_en;
        input  wdata;
        input  rdata;
    endclocking

    modport DRIVER (clocking driver_cb, input clk, reset);
    modport MONITOR (clocking monitor_cb, input clk, reset);
endinterface

```

Sabit kalıp (her interface'te aynı): `clocking ... endclocking`, `default input #1 output #1;`, `modport` yapısı, ve **yön kuralı** (DUT input → driver output; DUT output → driver input; monitor'da hepsi input).

DUT'a özel (biz yazdık): sinyal isimleri, genişlikleri (`[1:0]` , `[7:0]`) ve yönleri — hepsi design.sv'nin port listesinden geldi.

Neden clocking block? Sinyalleri saatle senkron sürüp örneklemek için. `input #1` = saat kenarından 1 birim önce örnekle; `output #1` = kenardan 1 birim sonra sür. Böylece yarış (race) olmadan çalışır.

7.3 — mem_seq_item.sv (transaction) — bir işlemi tarif eden paket [OBJECT]

```

class seq_item extends uvm_sequence_item;
    rand bit [1:0] addr;      // testbench üretir (DUT girişi) -> rand
    rand bit      wr_en;
    rand bit      rd_en;
    rand bit [7:0] wdata;
    bit [7:0] rdata;         // DUT döndürür (DUT çıkışı)      -> rand DEĞİL

    `uvm_object_utils_begin(seq_item)      // sınıf adı ile AYNI olmalı!
        `uvm_field_int(addr, UVM_ALL_ON)
        `uvm_field_int(wr_en, UVM_ALL_ON)
        `uvm_field_int(rd_en, UVM_ALL_ON)
        `uvm_field_int(wdata, UVM_ALL_ON)
    `uvm_object_utils_end

    function new(string name = "seq_item");
        super.new(name);
    endfunction

    constraint wr_rd_c { wr_en != rd_en; }    // aynı anda hem yaz hem oku olmasın
endclass

```

Sabit kalıp: `extends uvm_sequence_item`, `uvm_object_utils_begin/end`, `function new(string name)`, `super.new(name)`.

DUT'a özel (biz yazdık): hangi alanlar (`addr/wr_en/rd_en/wdata/rdata`), genişlikleri, hangisi `rand` (girişler `rand`, çıkış `rdata` `rand` değil), ve `constraint wr_rd_c { wr_en != rd_en; }` — her işlem YA yazma YA okuma olsun diye koyduğumuz kısıt (kendi tasarım kararımız).

Önemli — kaçınılan hata: sınıf adı (`seq_item`) ile `uvm_object_utils_begin(...)` içindeki ad **birebir** aynı olmalı. Bizde başta `mem_seq_item` yazılmıştı → "not defined" hatası; `seq_item` yapınca düzeldi.

7.4 — mem_sequence.sv (senaryo) — [OBJECT]

```
class mem_sequence extends uvm_sequence #(seq_item);
  `uvm_object_utils(mem_sequence)
  function new(string name = "mem_sequence");
    super.new(name);
  endfunction
  task body();
    repeat(10) begin          // 10 rastgele işlem üret
      `uvm_do(req)            // üret + randomize + sequencer'a yolla (tek satır)
    end
  endtask
endclass
```

Sabit kalıp: `extends uvm_sequence #(...`, `uvm_object_utils`, `task body()`, ``uvm_do(req)` (req hazır gelir, ayrıca tanımlanmaz).

Bizim revize ettiğimiz: `repeat(10)`. Başta `repeat(3)` idi — çok az işlem ürettiyordu ve son işlem (okuma) scoreboard'a ulaşamadan simülasyon bitiyordu. 10'a çıkarınca hem yazma hem okuma bolca göründü.

7.5 — mem_sequencer_item.sv (trafik memuru) — [COMPONENT, en kısa dosya]

```
class mem_sequencer_item extends uvm_sequencer #(seq_item);
  `uvm_component_utils(mem_sequencer_item)
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
endclass
```

Neredeyse tamamen sabit kalıp. İçine mantık yazılmaz; temel `uvm_sequencer` her işi yapar. DUT'a özel tek şey: `#(seq_item)` — hangi paketi taşıdığı. `build/run_phase` YOK.

7.6 — mem_driver.sv (süren el) — [COMPONENT] — EN ÇOK DUT'A ÖZEL parçalardan

```

`define DRIV_IF vif.DRIVER.driver_cb

class mem_driver extends uvm_driver #(seq_item);
  virtual mem_if vif;
  `uvm_component_utils(mem_driver)

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if(!uvm_config_db#(virtual mem_if)::get(this, "", "vif", vif)) // vif'i AL
      `uvm_fatal("NO_VIF", "virtual interface set edilmeli")
    endfunction

  task run_phase(uvm_phase phase);
    forever begin
      seq_item_port.get_next_item(req); // sequencer'dan paket al
      drive(); // pinleri sür
      seq_item_port.item_done(); // bitti sinyali
    end
  endtask

  virtual task drive();
    `DRIV_IF.wr_en <= 0;
    `DRIV_IF.rd_en <= 0;
    @(posedge vif.DRIVER.clk);
    `DRIV_IF.addr <= req.addr;

    if(req.wr_en) begin // === YAZMA ===
      `DRIV_IF.wr_en <= req.wr_en;
      `DRIV_IF.wdata <= req.wdata;
      @(posedge vif.DRIVER.clk);
      `DRIV_IF.wr_en <= 0; // işlem bitti -> wr_en'i indir
    end
    else if(req.rd_en) begin // === OKUMA ===
      `DRIV_IF.rd_en <= req.rd_en;
      @(posedge vif.DRIVER.clk);
      `DRIV_IF.rd_en <= 0;
      @(posedge vif.DRIVER.clk); // 1 saat gecikme (DUT register'lı okuma)
      req.rdata = `DRIV_IF.rdata;
    end
  endtask
endclass

```

Sabit kalıp: virtual mem_if vif;, build_phase'de config_db::get, run_phase'de get_next_item → drive → item_done döngüsü, `define DRIV_IF.

DUT'a özel (biz yazdık): drive() içindeki protokol — - Yazma dalı: wr_en / wdata sür, bir saat bekle, sonra wr_en'i indir (indirmek önemli: yoksa monitor aynı yazmayı iki kez sayar). - Okuma dalı: rd_en sür, bekle, indir, **bir saat daha bekle** (DUT okumayı 1 saat sonra verdiği için), sonra rdata'yı al.

<= (nonblocking) vs = : pinleri sürerken <=, DUT'tan değer okurken = kullanılır (req.rdata = ...).

7.7 — mem_monitor.sv (izleyen göz) — [COMPONENT] — burada da çok DUT'a özel

```

class mem_monitor extends uvm_monitor;
    virtual mem_if vif;
    uvm_analysis_port #(seq_item) item_collected_port;    // scoreboard'a GÖNDEREN port
    seq_item trans_collected;
    `uvm_component_utils(mem_monitor)

    function new(string name, uvm_component parent);
        super.new(name, parent);
        trans_collected    = new();                      // OBJECT -> new()
        item_collected_port = new("item_collected_port", this);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if(!uvm_config_db#(virtual mem_if)::get(this, "", "vif", vif))
            `uvm_fatal("NO_VIF", "virtual interface set edilmeli")
    endfunction

    task run_phase(uvm_phase phase);
        forever begin
            @(posedge vif.MONITOR.clk);
            wait(vif.MONITOR.monitor_cb.wr_en || vif.MONITOR.monitor_cb.rd_en); // işlem olsun
            trans_collected.addr = vif.MONITOR.monitor_cb.addr;
            trans_collected.wr_en = vif.MONITOR.monitor_cb.wr_en;
            trans_collected.rd_en = vif.MONITOR.monitor_cb.rd_en;

            if(vif.MONITOR.monitor_cb.wr_en)                // yazma ise wdata'yı yakala
                trans_collected.wdata = vif.MONITOR.monitor_cb.wdata;

            if(vif.MONITOR.monitor_cb.rd_en) begin          // okuma ise rdata'yı yakala
                @(posedge vif.MONITOR.clk);
                @(posedge vif.MONITOR.clk);                // 2 saat: rdata otursun (bkz. debug)
                trans_collected.rdata = vif.MONITOR.monitor_cb.rdata;
            end

            item_collected_port.write(trans_collected);    // scoreboard'a yolla
        end
    endtask
endclass

```

Sabit kalıp: `uvm_monitor` (dikkat: `#(seq_item)` YOK), `analysis_port`, constructor'da `new()` 'lar, `config_db::get`, `forever` döngü, sonda `item_collected_port.write(...)`.

DUT'a özel (biz yazdık): hangi sinyal ne zaman örneklenir. Özellikle okuma dalında `rdata` 'yı yakalamadan önce **iki** `@(posedge)` beklememiz — bu, projedeki en öğretici düzeltme (aşağıda).

Önemli fark (driver vs monitor): `monitor` `analysis_port` **new()** ile kurulur (OBJECT gibi), çünkü port factory'ye kaydedilmez. `trans_collected` de `new()` ile kurulur.

7.8 — mem_agent.sv — [COMPONENT] — üç parçayı toplar

```

class mem_agent extends uvm_agent;
    mem_driver          driver;
    mem_monitor          monitor;
    mem_sequencer_item sequencer;
    `uvm_component_utils(mem_agent)

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        driver      = mem_driver::type_id::create("driver", this);
        monitor      = mem_monitor::type_id::create("monitor", this);
        sequencer     = mem_sequencer_item::type_id::create("sequencer", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        driver.seq_item_port.connect(sequencer.seq_item_export);    // driver <-> sequencer
    endfunction
endclass

```

Neredeyse tamamen sabit kalıp. DUT'a özel değil; sadece hangi tipleri (`mem_driver` vb.) kullandığımız değişir.

- `build_phase`: alt bileşenleri `type_id::create` ile üret. (`create` + constructor birlikte çalışır: `create` fabrikayı çağırır, o da `new` 'i çağırır.)
- `connect_phase`: driver'ın `seq_item_port` 'unu sequencer'ın `seq_item_export` 'una bağla — paketin akacağı yol.

7.9 — mem_scoreboard.sv (hakem) — [COMPONENT] — asıl DOĞRULAMA burada

```

class mem_scoreboard extends uvm_scoreboard;
    bit [7:0] sc_mem [4];                                // referans (altın) bellek
    uvm_analysis_imp #(seq_item, mem_scoreboard) item_collected_export; // ALAN port
    `uvm_component_utils(mem_scoreboard)

    function new(string name, uvm_component parent);
        super.new(name, parent);
        item_collected_export = new("item_collected_export", this);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        foreach(sc_mem[i]) sc_mem[i] = 8'hFF;             // DUT reset'te FF yapıyordu
    endfunction

    function void write(seq_item pkt);                   // monitor her paket için çağırır
        if (pkt.wr_en) begin                             // YAZMA: referansa kaydet
            sc_mem[pkt.addr] = pkt.wdata;
            `uvm_info(get_type_name(),
                $sformatf("WRITE -> addr=%0d, wdata=0x%0h", pkt.addr, pkt.wdata), UVM_LOW)
        end
        else if (pkt.rd_en) begin                         // OKUMA: karşılaştır
            if (sc_mem[pkt.addr] == pkt.rdata)
                `uvm_info(get_type_name(),
                    $sformatf("MATCH -> addr=%0d, exp=0x%0h, dut=0x%0h",
                        pkt.addr, sc_mem[pkt.addr], pkt.rdata), UVM_LOW)
            else
                `uvm_error(get_type_name(),
                    $sformatf("MISMATCH -> addr=%0d, exp=0x%0h, dut=0x%0h",
                        pkt.addr, sc_mem[pkt.addr], pkt.rdata))
        end
    endfunction
endclass

```

Sabit kalıp: `uvm_scoreboard`, `uvm_analysis_imp` #(paket, kendi_sınıfı), constructor'da `imp`'i `new()` et, ve `write()` adında bir fonksiyon (`analysis_imp` `write` çağırır — isim tam "write" olmalı).

DUT'a özel (biz yazdık — bu projenin kalbi): - `sc_mem` = DUT belleğinin bir kopyası; `build_phase` 'de FF ile başlar (DUT reset'iyle aynı). - `write()` içi: yazma gelirse `sc_mem[addr] = wdata`; okuma gelirse `sc_mem[addr] == pkt.rdata` karşılaştır → eşitse MATCH (`uvm_info`), değilse

MISMATCH (uvm_error).

port farkı: monitor **analysis_port** (gönderen), scoreboard **analysis_imp** (alan). imp, gelen paketi senin write() fonksiyonuna teslim eder.

7.10 — mem_env.sv — [COMPONENT] — agent + scoreboard'u toplar

```
class mem_env extends uvm_env;
    mem_agent      agent;
    mem_scoreboard scoreboard;
    `uvm_component_utils(mem_env)

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        agent      = mem_agent::type_id::create("agent", this);
        scoreboard = mem_scoreboard::type_id::create("scoreboard", this);
    endfunction

    function void connect_phase(uvm_phase phase);
        // monitor'ün GÖNDEREN portu -> scoreboard'un ALAN portu
        agent.monitor.item_collected_port.connect(scoreboard.item_collected_export);
    endfunction
endclass
```

Neredeyse sabit kalıp. En kritik satır connect_phase : monitor'ün analysis_port 'unu scoreboard'un analysis_imp 'ine bağlar. Bu bağlantı sayesinde monitor'ün topladığı her paket scoreboard'un write() 'ına düşer.

7.11 — mem_base.sv (test) — [COMPONENT] — senaryoyu başlatır

```
class mem_base_test extends uvm_test;
    `uvm_component_utils(mem_base_test)
    mem_env      env;
    mem_sequence seq;

    function new(string name = "mem_base_test", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        env = mem_env::type_id::create("env", this);
    endfunction

    task run_phase(uvm_phase phase);
        seq = mem_sequence::type_id::create("seq"); // sequence OBJECT -> parent yok
        phase.raise_objection(this);               // "işim var, bitirme"
        seq.start(env.agent.sequencer);             // senaryoyu sequencer'da başlat
        #50;                                         // monitor/scoreboard son işlemleri sindirsin
        phase.drop_objection(this);                 // "işim bitti"
    endtask
endclass
```

Sabit kalıp: uvm_test , build_phase 'de env'i üret, run_phase 'de raise_objection → seq.start → drop_objection .

Önemli ayrıntılar: - seq = mem_sequence::type_id::create("seq") — sequence bir OBJECT olduğu için create 'e **parent verilmez** (component'lerde create("isim", this) idi; burada sadece create("seq")). - raise/drop_objection : objection kalkınca test biter. seq.start bittiği an düşürürsek monitor son paketi yakalayamadan simülasyon kapanır — o yüzden aralarına **#50** koyduk (bizim revize ettiğimiz kısım).

7.12 — tbench_top.sv (en üst) — [MODULE] — hepsini birleştirir

```

`include "uvm_macros.svh"
import uvm_pkg::*;

`include "mem_if.sv"
`include "mem_seq_item.sv"
`include "mem_sequence.sv"
`include "mem_sequencer_item.sv"
`include "mem_driver.sv"
`include "mem_monitor.sv"
`include "mem_agent.sv"
`include "mem_scoreboard.sv"
`include "mem_env.sv"
`include "mem_base.sv"

module tbench_top;
    bit clk;
    bit reset;
    always #5 clk = ~clk; // saat üret
    initial begin reset = 1; #5 reset = 0; end // reset üret

    mem_if intf(clk, reset); // interface örneği

    memory DUT ( // DUT örneği (portları intf'e bağlı)
        .clk(intf.clk), .reset(intf.reset),
        .addr(intf.addr), .wr_en(intf.wr_en), .rd_en(intf.rd_en),
        .wdata(intf.wdata), .rdata(intf.rdata)
    );

    initial begin
        uvm_config_db#(virtual mem_if)::set(null, "*", "vif", intf); // vif'i herkese DAĞIT
    end

    initial run_test(); // testi başlat (+UVM_TESTNAME ile)
endmodule

```

Sabit kalıp: ``include "uvm_macros.svh", import uvm_pkg::*; , saat/reset üretimi, DUT+interface örneği, config_db::set , run_test()`.

DUT'a özel: DUT port bağlantıları ve interface örneği.

`include sırası önemli: her sınıf, kendinden önce gelen tiplere dayandığı için include sırası aşağıdan-yukarı hiyerarşiyle aynıdır (if → seq_item → sequence → ... → base).

config_db köprüsü: top'ta `set(null, "*", "vif", intf)` ile interface dağıtılır; driver ve monitor `build_phase`'de `get(this, "", "vif", vif)` ile alır. Etiket ("vif") iki tarafta da aynı olmalı.

7.13 — DEBUG GÜNLÜĞÜ (mentör görüşmesi için altın değerinde)

Testbench ilk seferde çalışmadı; karşılaştığımız her sorunu ve çözümünü **anlatabilmen** çok kıymetli. Sırasıyla:

- PortRedefinition (design.sv):** `rdata` iki kez tanımlıydı → port listesinde `output reg` yapıp gövdedeki ayrı `reg` 'i sildik.
- seq_item not defined:** sınıf adı `seq_item` ama factory kaydı `mem_seq_item` idi → ikisini eşitledik.
- Scoreboard boş write() :** karşılaştırma kodunu (yaz→kaydet, oku→karşılaştır) yazdık, `endclass` ekledik.
- Hiç okuma yok, sadece yazma:** `repeat(3)` çok azdı ve test erken bitiyordu → `repeat(10)` yaptık ve `mem_base` 'de `drop_objection` 'dan önce `#50` bekleme ekledik. Böylece okumalar (MATCH mesajları) göründü.
- İlk okumada sahte MISMATCH (addr=2, exp=ff, dut=0):** DUT okuması **1 saat gecikmeli** olduğu için monitor `rdata` 'yı oturmadan (başlangıç değeri 0) örnekliliyordu. Aynı adres ilerde FF ile MATCH veriyordu — yani scoreboard mantığı değil, **zamanlama** hatasıydı. Çözüm: monitörün okuma dalında `rdata` 'yı almadan önce **iki** @(posedge) bekletmek. Sonuç: **UVM_ERROR: 0**.

Ana ders: Zamanlama hataları tahminle değil, sinyalleri (clk, rd_en, rdata) izleyerek çözülür. DUT'un register'lı okuması (1 saat gecikme) monitörün örnekleme anını belirler.

7.14 — Çalıştırma: filelist ve komut

filelist.f (mem_vf kökünden çalışır):

```
+incdir+sim
src/design.sv
sim/tbench_top.sv
```

tbench_top.sv zaten bütün class'ları `include ettiği için listeye SADECE design.sv + tbench_top.sv yazılır (hepsini eklersek "already defined" olur).

DSim komutu:

```
dsim -sv -uvm 1.2 -timescale 1ns/1ns -f filelist.f -top tbench_top +UVM_TESTNAME=mem_base_test +acc+b -waves waves.mxd
```

Başarı işareti: log'da WRITE -> ..., MATCH -> ... satırları ve en sonda UVM_ERROR : 0 .

7.15 — Tek bakışta: hangi dosya ne kadar "bizim", ne kadar "kalıp"?

Dosya	Sabit kalıp	Bizim yazdığımız (DUT'a özel)
interface	clocking/modport/yön kuralı	sinyal isim/genişlik
seq_item	class/utils/new	alanlar, rand seçimi, constraint
sequence	body/ uvm_do	repeat(10) senaryosu
sequencer	neredeyse tamamı	sadece #(seq_item)
driver	get_next_item/item_done, config_db	drive() protokolü, wr_en indirme, okuma gecikmesi
monitor	analysis_port + döngü	hangi sinyal ne zaman; okumada 2 saat bekleme
agent	neredeyse tamamı	kullanılan tip adları
scoreboard	analysis_imp + write() iskeleti	sc_mem, FF init, karşılaştırma mantığı
env	neredeyse tamamı	connect satırı
test	raise/start/drop	#50 drain
top	include/config_db/run_test	DUT bağlantıları

En çok emek isteyen (DUT'a en özel): driver, monitor, scoreboard. **Neredeyse hiç değişmeyen:** sequencer, agent, env.